Міністерство освіти і науки України
Національний технічний університет України
«Київський політехнічний інститут імені Ігоря Сікорського»
Факультет інформатики та обчислювальної техніки
Кафедра обчислювальної техніки

Лабораторна робота №2

з дисципліни
«Об'єктно-орієнтоване програмування»

на тему

«Розробка графічного редактора об'єктів»

Виконала:
студентка групи ІМ-55
Сітковська Аліна
Номер в списку - 27

Перевірив:
Рекечинський Д. О.

Київ 2026

Завдання:

1. Створити у середовищі розробки новий проект з ім'ям Lab2.
2. Скомпілювати проект і отримати виконуваний файл програми.
3. Перевірити роботу програми. За потреби налагодити програму.
4. Проаналізувати та прокоментувати результати та вихідний текст програми.
5. Оформити звіт.

Завдання згідно варіанту (Ж = 27):

Розмір масиву: N = Ж + 100 = 127.

Тип масиву: Ж mod 3 = 0 → динамічний масив (у Python — список, що росте методом append() у міру додавання об'єктів, без попереднього виділення N елементів).

«Гумовий» слід: Ж mod 4 = 3 → пунктирна лінія чорного кольору.

Прямокутник, ввід: Ж mod 2 = 1 → від центру до одного з кутів.

Прямокутник, відображення: Ж mod 5 = 2 → чорний контур з кольоровим заповненням.

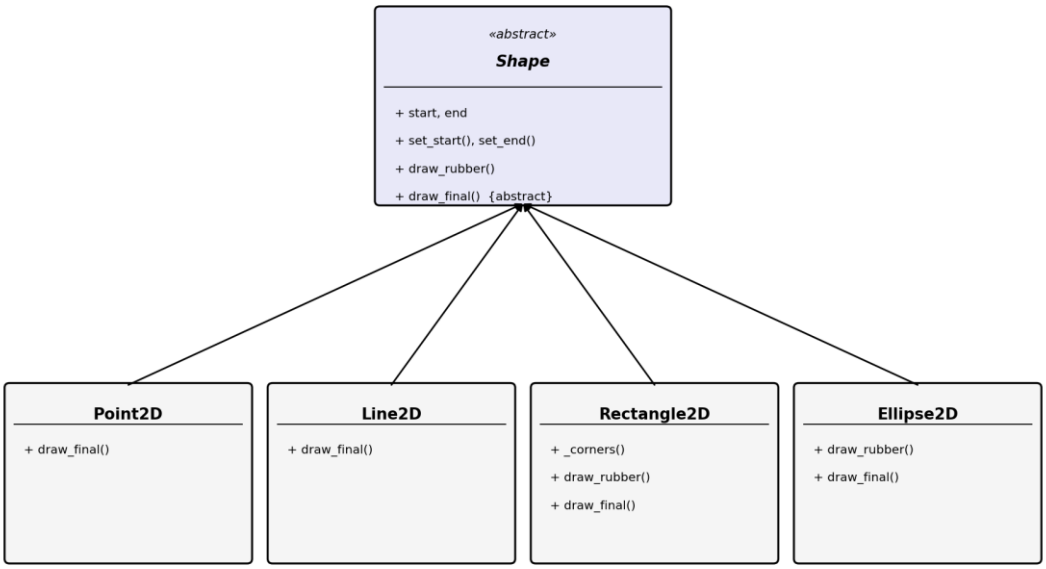
Прямокутник, колір заповнення: Ж mod 6 = 3 → рожевий.

Еліпс, ввід: Ж mod 2 = 1 → по двом протилежним кутам охоплюючого прямокутника.

Еліпс, відображення: Ж mod 5 = 2 → чорний контур без заповнення.

Позначка типу об'єкта: Ж mod 2 = 1 → у заголовку вікна.

Схема успадкування класів:



Shape — абстрактний базовий клас (містить спільні дані start/end та спільну реалізацію "гумового" сліду). Point2D, Line2D, Rectangle2D та Ellipse2D успадковують Shape і перевизначають метод draw_final(), а Rectangle2D і Ellipse2D додатково перевизначають draw_rubber() під свою форму попереднього перегляду.

Код:

shape.py:

```
from abc import ABC, abstractmethod


class Shape(ABC):
    RUBBER_COLOR = "black"
    RUBBER_DASH = (4, 2)

    def __init__(self):
        self.start = None
        self.end = None

    def set_start(self, point):
        self.start = point

    def set_end(self, point):
        self.end = point

    @abstractmethod
    def draw_final(self, canvas):
        raise NotImplementedError

    def draw_rubber(self, canvas, current_point, tag="rubber"):
        canvas.delete(tag)
        if self.start:
            canvas.create_line(
                self.start[0], self.start[1],
                current_point[0], current_point[1],
                fill=self.RUBBER_COLOR, dash=self.RUBBER_DASH, width=1, tags=tag
            )
```

point.py:

```
from shape import Shape


class Point2D(Shape):
    RADIUS = 3

    def draw_rubber(self, canvas, current_point, tag="rubber"):
        canvas.delete(tag)

    def draw_final(self, canvas):
        x, y = self.end if self.end else self.start
        canvas.create_oval(
            x - self.RADIUS, y - self.RADIUS,
            x + self.RADIUS, y + self.RADIUS,
            outline="black", fill="black"
        )
```

line.py:

```
from shape import Shape


class Line2D(Shape):
    def draw_final(self, canvas):
        canvas.create_line(
            self.start[0], self.start[1],
            self.end[0], self.end[1],
            fill="black", width=1
        )
```

rectangle.py:

```
from shape import Shape


class Rectangle2D(Shape):
    FILL_COLOR = "pink"

    def _corners(self, current_point):
        cx, cy = self.start
        dx = abs(current_point[0] - cx)
        dy = abs(current_point[1] - cy)
        return (cx - dx, cy - dy, cx + dx, cy + dy)

    def draw_rubber(self, canvas, current_point, tag="rubber"):
        canvas.delete(tag)
        if self.start:
            x1, y1, x2, y2 = self._corners(current_point)
            canvas.create_rectangle(x1, y1, x2, y2, outline=self.RUBBER_COLOR,
dash=self.RUBBER_DASH, tags=tag)

    def draw_final(self, canvas):
        x1, y1, x2, y2 = self._corners(self.end)
        canvas.create_rectangle(x1, y1, x2, y2, outline="black", fill=self.FILL_COLOR)
```

elipse.py:

```
from shape import Shape


class Ellipse2D(Shape):
    def draw_rubber(self, canvas, current_point, tag="rubber"):
        canvas.delete(tag)
        if self.start:
            canvas.create_rectangle(
                self.start[0], self.start[1],
                current_point[0], current_point[1],
                outline=self.RUBBER_COLOR, dash=self.RUBBER_DASH, tags=tag
            )

    def draw_final(self, canvas):
        x1, y1 = self.start
        x2, y2 = self.end
        canvas.create_oval(x1, y1, x2, y2, outline="black", fill="")
```

lab2.py:

```
import tkinter as tk
from tkinter import messagebox

from point import Point2D
from line import Line2D
from rectangle import Rectangle2D
from elipse import Ellipse2D

N = 127

SHAPE_CLASSES = {
    "point": Point2D,
    "line": Line2D,
    "rectangle": Rectangle2D,
    "ellipse": Ellipse2D,
}
SHAPE_NAMES_UA = {
    "point": "Крапка",
    "line": "Лінія",
```

```
    "rectangle": "Прямокутник",
    "ellipse": "Еліпс",
}


class GraphicEditorApp:
    def __init__(self, root):
        self.root = root
        self.pcshape = []
        self.current_type = "rectangle"
        self.drawing = False
        self.current_shape = None

        self._build_menu()
        self._build_canvas()
        self._update_title()

    def _build_menu(self):
        menubar = tk.Menu(self.root)

        file_menu = tk.Menu(menubar, tearoff=0)
        file_menu.add_command(label="Новий", command=self.on_new)
        file_menu.add_separator()
        file_menu.add_command(label="Вихід", command=self.root.quit)
        menubar.add_cascade(label="Файл", menu=file_menu)

        objects_menu = tk.Menu(menubar, tearoff=0)
        self.type_var = tk.StringVar(value=self.current_type)
        for key in ("point", "line", "rectangle", "ellipse"):
            objects_menu.add_radiobutton(
                label=SHAPE_NAMES_UA[key],
                variable=self.type_var,
                value=key,
                command=self.on_type_change,
            )
        menubar.add_cascade(label="Об'єкти", menu=objects_menu)

        help_menu = tk.Menu(menubar, tearoff=0)
        help_menu.add_command(label="Про програму", command=self.on_about)
        menubar.add_cascade(label="Довідка", menu=help_menu)

        self.root.config(menu=menubar)

    def _build_canvas(self):
        self.canvas = tk.Canvas(self.root, width=700, height=500, bg="white")
        self.canvas.pack(fill=tk.BOTH, expand=True)
        self.canvas.bind("<ButtonPress-1>", self.on_mouse_down)
        self.canvas.bind("<B1-Motion>", self.on_mouse_move)
        self.canvas.bind("<ButtonRelease-1>", self.on_mouse_up)

    def _update_title(self):
        self.root.title(f"ООР_Lab2 (Варіант 27) - {SHAPE_NAMES_UA[self.current_type]}")

    def on_type_change(self):
        self.current_type = self.type_var.get()
        self._update_title()

    def on_new(self):
        self.canvas.delete("all")
        self.pcshape = []

    def on_about(self):
        messagebox.showinfo(
            "Про програму",
            "Лабораторна робота №2\nГрафічний редактор об'єктів\nВаріант 27",
```

```
        )

    def on_mouse_down(self, event):
        if len(self.pcshape) >= N:
            messagebox.showwarning("Увага", "Масив об'єктів переповнено")
            return
        cls = SHAPE_CLASSES[self.current_type]
        self.current_shape = cls()
        self.current_shape.set_start((event.x, event.y))
        self.drawing = True

    def on_mouse_move(self, event):
        if self.drawing and self.current_shape:
            self.current_shape.draw_rubber(self.canvas, (event.x, event.y))

    def on_mouse_up(self, event):
        if self.drawing and self.current_shape:
            self.canvas.delete("rubber")
            self.current_shape.set_end((event.x, event.y))
            self.current_shape.draw_final(self.canvas)
            self.pcshape.append(self.current_shape)
            self.current_shape = None
            self.drawing = False


if __name__ == "__main__":
    root = tk.Tk()
    app = GraphicEditorApp(root)
    root.mainloop()
```

Скріншоти:

[тут — скріншот головного вікна з меню "Об'єкти" та намальованими фігурами (крапка, лінія, прямокутник, еліпс)]

[тут — скріншот, де видно рожеве заповнення прямокутника та контур еліпса без заповнення]

[тут — скріншот заголовка вікна з назвою поточного обраного типу об'єкта]

Висновок

На даній лабораторній роботі я отримала навички використання інкапсуляції, абстракції типів, успадкування та поліморфізму на основі класів, запрограмувавши простий графічний редактор об'єктів. Абстрактний клас Shape задає спільний інтерфейс (set_start, set_end, draw_rubber, draw_final), а кожен з чотирьох класів-нащадків (Point2D, Line2D, Rectangle2D, Ellipse2D) реалізує власну версію малювання — це і є поліморфізм: редактор працює з об'єктами через єдиний інтерфейс Shape, не знаючи заздалегідь, яка саме фігура буде намальована. Побудова об'єктів у динамічний список замість фіксованого масиву дозволила не обмежувати кількість фігур наперед, а винесення спільної логіки (збереження координат, малювання "гумової" лінії) у базовий клас позбавило від дублювання коду в кожному з нащадків.